Meeting Scheduler Agent

An intelligent, browser-based scheduling system combining rule-based AI with `zlib` compression — fully offline, no API key required.

Stack: Streamlit • Python
Compression: zlib + Base64
License: MIT **API Key:** None

share.streamlit.io

Smart Calendar | AI Scheduling | Conflict Detection | Analytics | Compression | Export

1 Overview

The **Meeting Scheduler Agent** is a browser-based scheduling system built with Streamlit, combining a rule-based AI engine with `zlib` data-compression to manage calendars, detect conflicts in real time, and surface optimal meeting slots — entirely offline with no external dependencies.

Key Highlights

- **No API key** — fully offline, rule-based scheduling engine.
- **60–70% payload reduction** via `zlib` + Base64 encoding.
- **Scoring AI** ranks available slots on a 0–100 scale.
- **Real-time conflict detection** flags overlaps instantly on entry.
- Clean **tabbed Streamlit UI** with analytics and export built in.

2 Features

Feature	Description
Calendar Mgmt	Add, view, and delete meetings with automatic overlap detection.
AI Scheduling	Multi-factor scoring finds the most productive available slots.
Preferences	Configure work hours, buffer time, duration, and daily meeting cap.
Conflict Detection	Instant highlighting of scheduling overlaps on every new entry.
Analytics	Meeting load patterns, time distribution, and compression stats.
Compression	`zlib` level-9 + Base64 cuts JSON payload by 60–70%.
Export	One-click download of the full calendar dataset.

3 Quick Start

Listing 1: Setup and launch

```
# Create and activate a virtual environment
python -m venv venv
source venv/bin/activate        # macOS/Linux
# .\venv\Scripts\activate       # Windows

pip install -r requirements.txt # Install dependencies
streamlit run app.py             # Launch at localhost:8501
```

4 User Guide

4.1 Preferences (Sidebar)

Set timezone, work hours, meeting duration, buffer time between meetings, and daily meeting limit, then click **"Save Preferences"** to apply.

4.2 Adding a Meeting

In the **Add Meeting** tab, enter the title, attendees (comma-separated emails), date/time, duration, and an optional description. Click **"Add Meeting"** — conflicts are flagged instantly.

4.3 AI Scheduler

In the **AI Scheduler** tab, provide the meeting title, attendee count, duration, search date range, and priority level. Click **"Find Best Times"** to receive a scored, ranked list of suggested slots, then book your preferred option.

4.4 Calendar & Analytics

The **Calendar** tab lists all upcoming meetings in chronological order. The **Analytics** tab shows meeting load patterns, daily time distribution, and compression statistics.

5 How It Works

5.1 Scheduling Algorithm

Every candidate slot is scored on a **0–100 scale** across three factor groups. Slots are ranked by score and returned as suggestions.

5.1.1 Time-of-Day & Day-of-Week Scoring

Factor	Score	Rationale
9 AM – 11 AM	**+20**	Peak focus; ideal for deep-work meetings.
2 PM – 4 PM	**+15**	Post-lunch recovery; good for collaboration.
Tue – Thu	**+10**	Mid-week peak availability.
Meetings ≥60 min (AM)	**+10**	Morning bonus for long sessions.
After 4 PM	¬10	Declining energy and engagement.
Monday	¬5	Catch-up tasks reduce availability.
12 PM – 1 PM	¬15	Lunch hour; low attendance.
Friday afternoon	¬15	Pre-weekend; low engagement.

5.2 Data Compression

Calendar JSON is serialised, `zlib`-compressed at level 9, and Base64-encoded — yielding a **60–70% size reduction** for faster payloads and lower bandwidth usage.

Listing 2: Compression and decompression

```python
import zlib, base64, json

def compress_calendar(data: dict) -> str:
    raw  = json.dumps(data).encode("utf-8")
    comp = zlib.compress(raw, level=9)
    return base64.b64encode(comp).decode("ascii")

def decompress_calendar(blob: str) -> dict:
    comp = base64.b64decode(blob.encode("ascii"))
    raw  = zlib.decompress(comp)
    return json.loads(raw.decode("utf-8"))
```

Compression Results

- **Input:** Full JSON with all meeting fields and metadata.
- **Output:** Base64-encoded `zlib` blob, safe for transport.
- **Savings:** 60–70% size reduction on representative datasets.
- **Benefit:** Faster API calls, lower bandwidth, reduced storage.

6 Project Structure

Listing 3: Repository layout

```
meeting-scheduler-agent/
|-- app.py                # Streamlit UI and tabbed navigation
|-- scheduler_agent.py    # AI scoring engine and slot finder
|-- calendar_manager.py   # Data management and compression
|-- requirements.txt      # Python dependencies
|-- README.md
```

- `calendar_manager.py` — add/delete meetings, conflict checks, availability queries, compression pipeline, calendar summaries.
- `scheduler_agent.py` — slot enumeration, multi-factor scoring, rescheduling suggestions, daily load enforcement.
- `app.py` — tabbed Streamlit interface, conflict alerts, analytics charts, preference management, and data export.

7 Deployment

Listing 4: Push to GitHub and deploy

```
git init && git add .
git commit -m "Initial commit: Meeting Scheduler Agent"
git remote add origin https://github.com/USER/meeting-scheduler-agent.git
git push -u origin main
```

Go to `https://share.streamlit.io`, sign in with GitHub, click **"New app"**, select your repo and `app.py` as the entry point, and click **Deploy**. Your app will be live at:

`https://meeting-scheduler-agent-xxxxx.streamlit.app`

Deployment Note

No environment variables or database configuration are required. The app runs entirely on in-memory data.

8 Example Use Cases

Common Workflows

1. **Personal Calendar Management** — track all meetings centrally with conflict protection and export.
2. **Team Scheduling** — find windows that respect each member's preferences and existing commitments.
3. **Interview Scheduling** — optimise candidate slots, avoiding back-to-back sessions.
4. **Client Meetings** — surface professional, high-scoring time windows aligned with peak engagement hours.
5. **Recurring Meetings** — consistently book weekly syncs at the same optimal slot each cycle.

9 Roadmap

Feature	Description
Email Integration	Two-way sync with Google Calendar and Outlook via OAuth 2.0.
Recurring Meetings	Daily, weekly, bi-weekly, and monthly recurrence rules.
Team Availability	Aggregate free/busy data across multiple team members.
Video Conferencing	Auto-generate Zoom, Teams, or Meet links on booking.
Meeting Reminders	Push and email alerts sent before scheduled meetings.
Timezone Conversion	Auto-convert times for cross-timezone participants.
Meeting Templates	Save and reuse titles, durations, and attendee lists.

10 Contributing & License

Contributions are welcome. Fork the repository, create a feature branch (`git checkout -b feature/name`), commit your changes, and open a Pull Request against `main` with a clear description and any relevant testing notes. Bug reports and feature requests can be submitted as GitHub issues.

This project is released under the **MIT License** — free to use, modify, and distribute with attribution.

Built with **Streamlit** · MIT License · February 12, 2026

For questions, open an issue on GitHub.